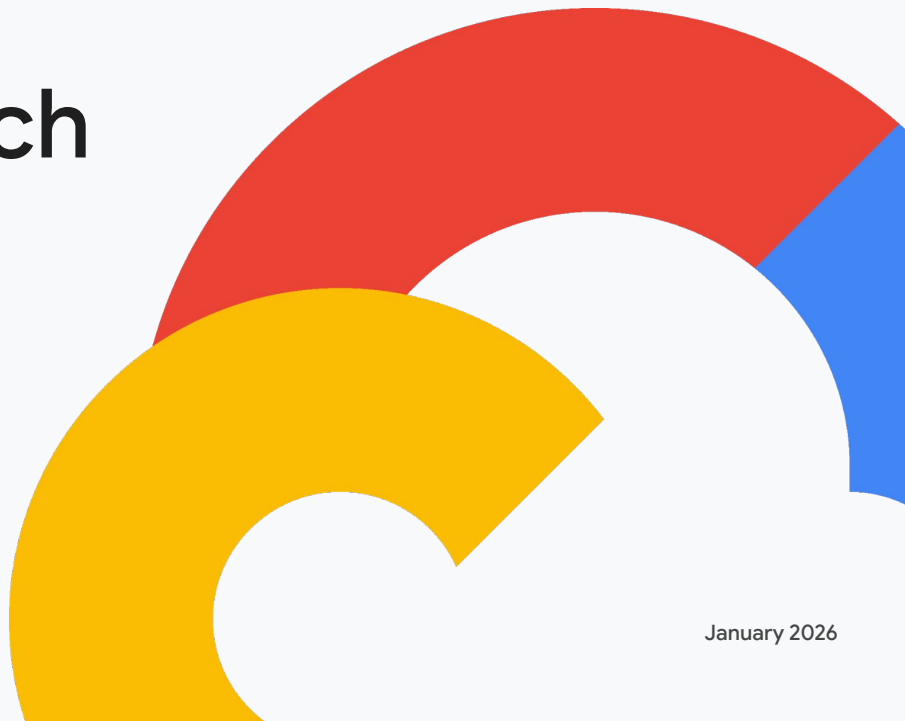


Bringing Vector Search to OLTP



Generative AI presents numerous opportunities for businesses to reinvent how things are done, and how data are turned into knowledges.

~\$30 billions GenAI market size 2026 - [Grand View Research](#)

MySQL customers need help building GenAI capabilities using datastore they are familiar with

MySQL customers want to have a single database that can serve all of their business needs including GenAI transformation

Main challenges



Create Embeddings

Customers can generate embeddings for their existing data without the need to go outside of the database.



Vector Index

Vector indexes are managed and utilized as normal indexes, supporting full transactional semantics and crash safety.



SQL-native Search

Performance ANN searches in a native SQL way. Supporting both relational and vector predicates.

ANN Engine - ScaNN

ScaNN (Scalable Nearest Neighbors)

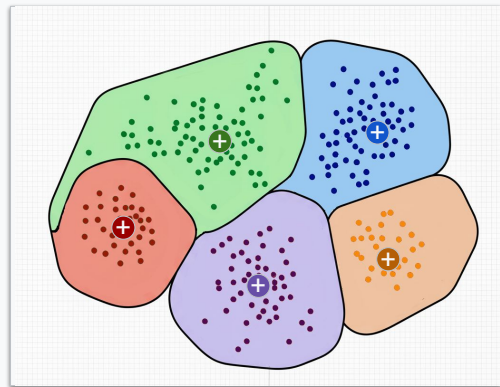
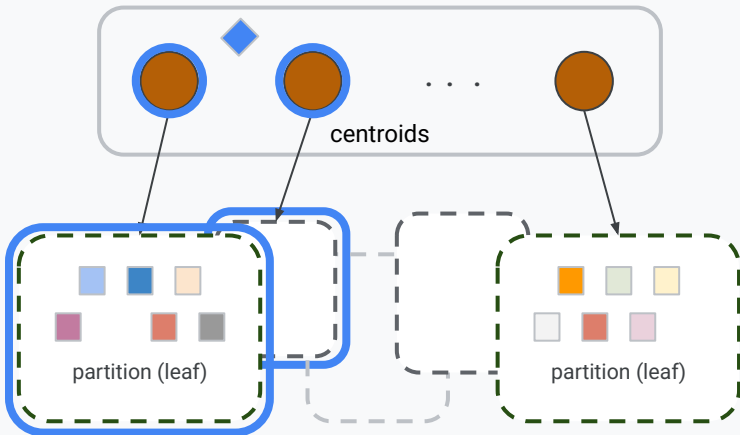


ann-benchmark

state-of-the-art ANN search engine powering many planet scale Google products

ScaNN Index -> K-Means partitioning

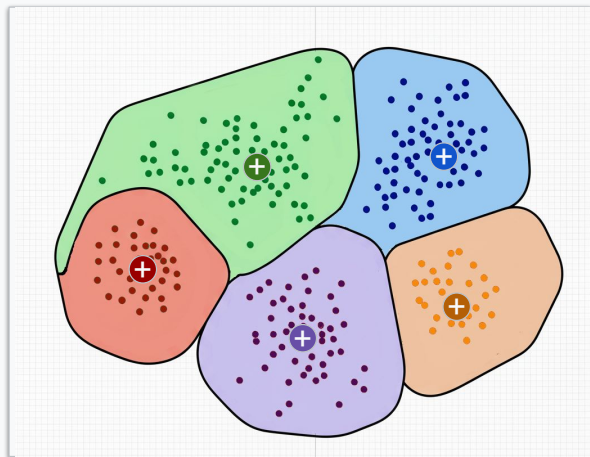
- Build = train (compute centroids) + insert (add data points - vectors)
 - Each partition (leaf) has a certain number of vectors
 - Vectors can be quantized (compressed)



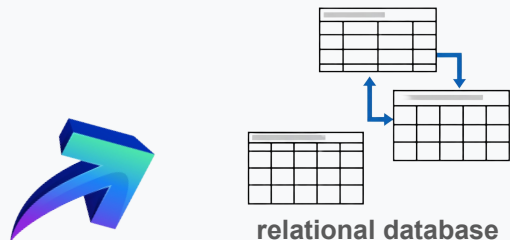
ANN Search On ScaNN Index

- Compare the distance of centroids (brown circle) and the query vector (blue diamond), to find a set of partitions that are **close enough**.
- Scan through data points in those selected partitions to find n closest neighbors

Challenges - Data Format & Software Architecture



vector index

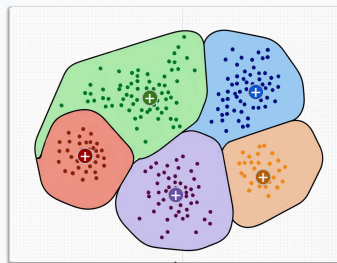


Vector Search Engine	Relational Database
Dedicated Compute	Multi-tasking
Single data, dedicated data type	Flexible data types, compounded structure
No state bookkeeping	MVCC (Multiple Version Concurrency Control)
Preloaded data	R/W data, Crash Safety

Cloud SQL for MySQL - Bridging ANN & SQL

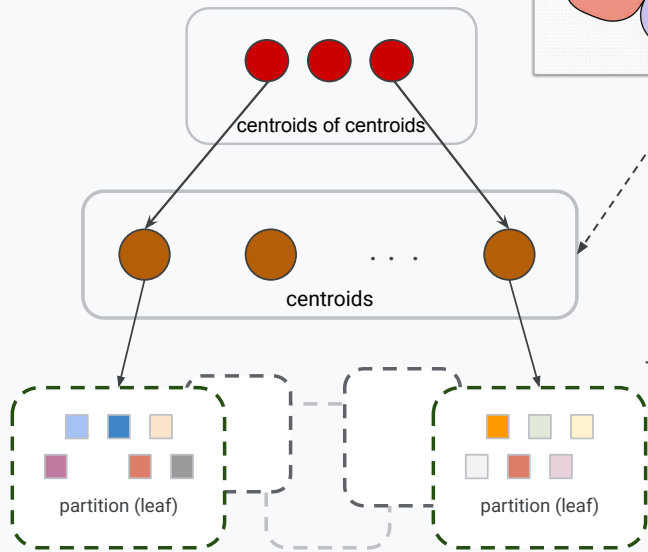
Scaling:

- More vectors = More partitions
- More levels



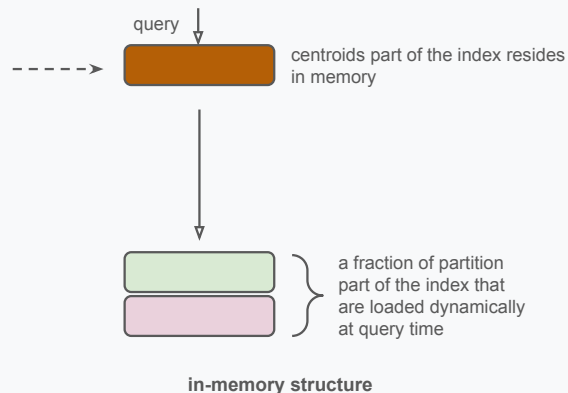
Overall Cost:

- Scan all centroids to find partitions that are close enough
- Scan the selected partitions to find the nearest neighbors

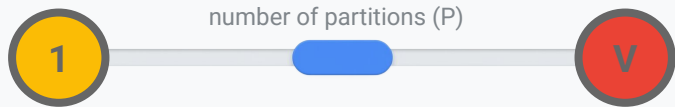


partition	vector ID	vector data
-1	null	centroids
0	0	...
0	1	...
...		
k	m	...
k	n	...

persistence



Balancing Throughput and Build Time



1: K-Means partitioning reduces to brute force

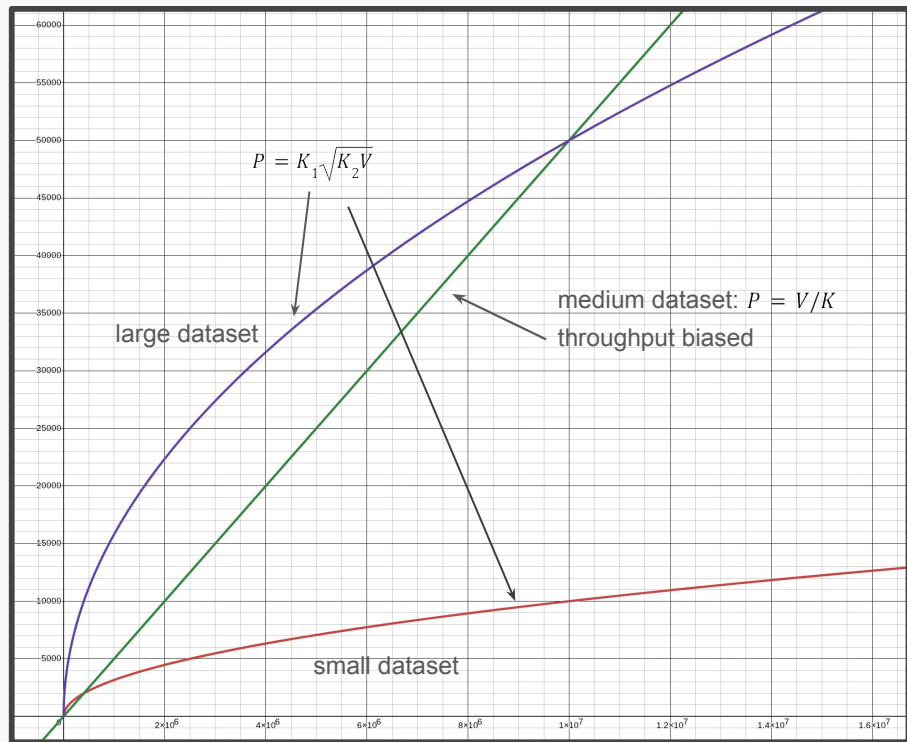
V: Brute force (centroids reduces to vectors)

The Balanced Configuration

- Search cost function $C(P)$ can be formulated as

$$C(P) = m * P + n * \frac{V}{P}$$
 (one vertex)
- The vertex is where $C'(P) = 0$, which gives us

$$P \propto \sqrt{V}$$



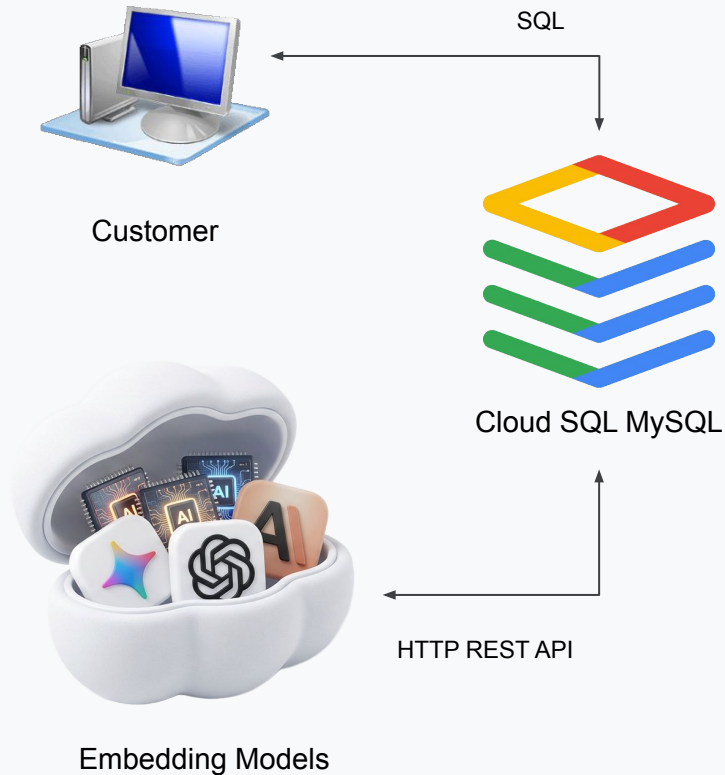
Generating Embeddings

Customer

- Customers want to compute embeddings directly from SQL, without external pipelines.

Model Endpoint Management

- Allows forwarding requests to models from different providers to generate embeddings
- Customers can get online predictions from models hosted in Google (Vertex AI) and other providers such as OpenAI, Anthropic, etc.
- Embeddings are extracted from the model results.



SQL-native Vector Management

Imagine an eCommerce scenario: product recommendation system for toys

- **Create Table**
 - `create table toys (id int primary key, ..., specs varchar(3000), emb vector(3072) using varbinary);`
- **Add Vector Column**
 - `alter table toys add column emb vector(3072) using varbinary;`
- **Populate Embedding Data**
 - `update toys set emb = mysql.ml_embedding('gemini-embedding-001', specs);`
 - `insert into toys values(1, "stuffed dragon", string_to_vector('[1,2,3,...]'));`
- **Create Vector Index**
 - `create vector index toy_specs_idx on toys (emb) distance_measure=cosine`



SQL-native Vector Search

ANN - Nearest Neighbor Search

- `approx_distance(<col_name>, <query_vector>, <distance_measure_option>)`
- In **ORDER BY** clause
`select id from toys order by approx_distance(emb, @stuffed_dragon, 'distance_measure=cosine') limit 10;`
- In **SELECT** list
`select id, approx_distance(emb, @stuffed_dragon, 'distance_measure=cosine') similarity from toys order by similarity limit 10;`

KNN

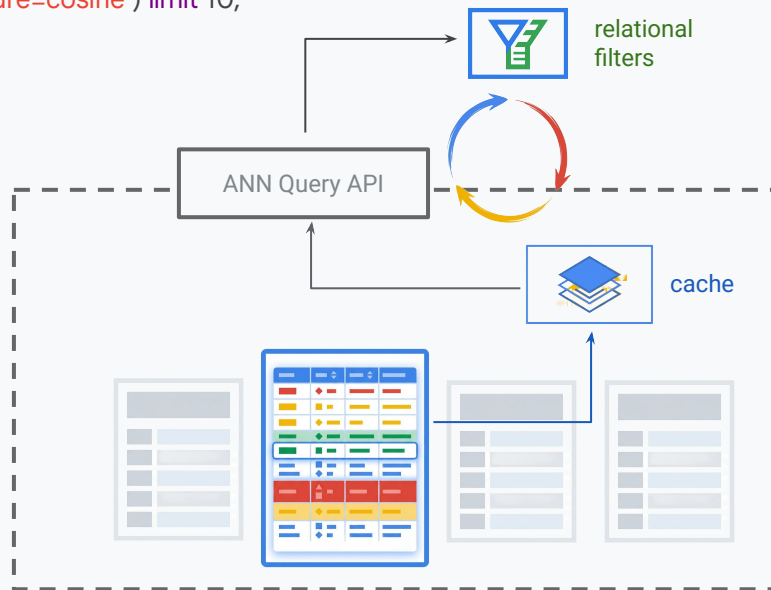
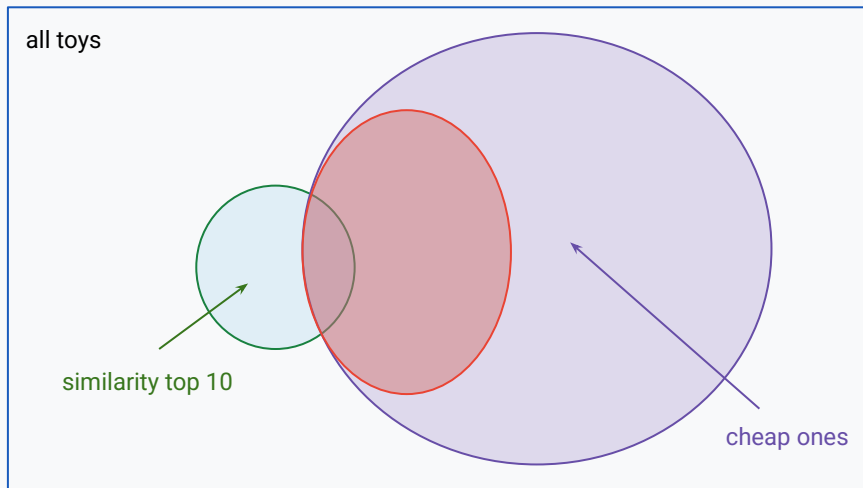
- If explicitly specifies KNN function
`select id from toys order by cosine_distance(emb, @stuffed_dragon, 'distance_measure=cosine') limit 10;`
- If there is no vector index
`select id from toys order by approx_distance(emb, @stuffed_dragon, 'distance_measure=cosine') limit 10;`
- If the desired distance measure algorithm is not what the index has been built on
`select id from toys order by approx_distance(emb, @stuffed_dragon, 'distance_measure=l2_squared') limit 10;`



Hybrid Search - ANN Iteration Pushdown

Hybrid Search - Search with both relational and ANN filters

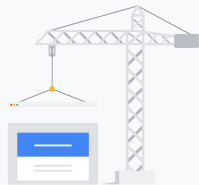
```
select id from toys  
  where price < 100  
  order by approx_distance(emb, @stuffed_dragon, 'distance_measure=cosine') limit 10;
```



Highlights

Feature & Metrics	Cloud SQL Implementation
Dataset Size (soft limit)	N/A (<i>billions</i> in a single instance in internal tests)
Vector Index Storage	On disk
Crash Safety	Yes
Persistence	As normal DB objects (tables)
MVCC	Fully transactional
Generating vector embeddings	Yes , through Model Endpoint Management
Vector Index DDLs	Regular DDL syntax
Optimizer aware of vector index	Yes
Optimizer choose between KNN or ANN	Yes
Hybrid Search: ANN & SQL	Iterative Search

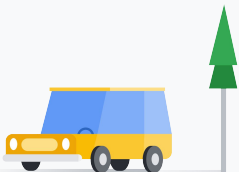
Resources



Guide

Look into Cloud SQL for MySQL documentation

- [Vector Search](#)
- [Integrate Cloud SQL with Vertex AI](#)



Test Drive

Create a MySQL 8.0.36+ instance in Cloud SQL

`cloudsql_vector=on`

- [Free Trial Instance](#)



Contact

Feel free to reach out to me for any questions, feedback, etc.

- zhoushu@google.com
- [linkedin.com/in/shuzhou](https://www.linkedin.com/in/shuzhou)